

Vibe Coding从入门到精通

核心 workflow

在上一课跑通第一个项目之后，你可能觉得Vibe Coding也就这样——写代码、确认权限、看结果。但日常用下来，真正拉开效率差距的是几个核心工作模式。这一课把它们拆开讲解。

Plan模式

先想清楚再动手Plan Mode

「一个好的计划真的很重要。」我自己大多数会话都从Plan模式开始。

Plan模式的作用很直接：让AI Agent只规划、不执行。它会告诉你它打算怎么做，但不会动你的代码、不会装包、不会运行命令。你们来回讨论方案，确认了再放手让它去做。

如何进入Plan模式

在codebuddy（其他的AI开发工具类似）的输入框中，选择对话框下方的下拉框。你会看到界面切换到Plan模式。此时的行为会改变：



- 它可以读取文件来理解代码，但不会修改任何文件
- 它会给出详细的实现方案，包括要改哪些文件、怎么改
- 你可以反复讨论、修改方案

Plan模式的黄金 workflow

1、Plan模式下描述需求，来回讨论。

进入Plan模式，描述你的需求。AI给出方案后，你可以说「第三步换个方式」「这里的库用XXX替换」，反复调整。

现在使用AI开发，有个问题，你并没有明确的需求，明确需求表示：功能列表明确，每个功能细节明确，页面的UI要求一致。但是自己一般来说AI给你了一个差不多的产品，你简单迭代就验收了。

2、用编辑器写一份详细的执行指令

方案大致满意后，会在你的默认编辑器（一般是markdown格式）中打开方案。你可以在新建一个修改意见文件，里写一份完整的执行指令，把讨论中确认的方案细节、约束条件都写进去，保存后内容回到 AI plan 的输入框，作为下一步的prompt发出。

3、切换到执行模式，开始执行

计划确认后，切回正常模式-Craft。因为计划已经讨论充分，你可以放心地让AI Agent一次性执行完成，不需要逐步确认。

这个流程的精髓在于：把纠结放在Plan阶段解决完，执行阶段一气呵成。边做边改、反复返工是最浪费tokens的用法。

什么时候该用Plan模式

用Plan模式	跳过Plan,直接做
你不确定实现方案	改一行代码、修个bug
需要改动多个文件	加一行console.log调试
你对这块代码不熟悉	你很清楚要改什么
重构或架构级别的变更	跑个测试、装个包
第一次接触一个新项目	重复性的日常操作

核心建议

一条实用判断标准：如果你需要跟同事解释才能让他做的任务，就值得用Plan模式。如果你能用一句话说清的任务，直接做就行。

执行模式（Craft）

更安全的自动驾驶

在完成Plan规划后可以开始执行模式，输入执行要求即可开始执行，在执行过程中会要部分用户授权操作，一般是对文件的修改和删除操作命令，用户需要检查核对命令，点击授权，避免全部授权的操作，每个授权操作需要人工确认。

典型事故案例

- **Pocket OS“9秒删库”事件（2026年4月）**
 - AI编程智能体（Cursor + Claude Opus 4.6）在预发布环境处理凭据问题时，**误将生产数据库一并删除。**
 - 原因：AI找到一个具有全权限的API令牌，直接调用删除命令，**无确认机制**，且备份与生产数据同存储卷，导致**95%数据永久丢失**，公司最终破产。
- **Claude CLI清空Mac主目录（2025年12月）**
 - 因执行命令 `rm -rf ~/`，**递归删除整个用户目录**（含桌面、文档、钥匙串等）。
- **Google Antigravity误删D盘（2025年12月）**
 - AI在“Turbo模式”下自动执行缓存清理，**错误指向D盘根目录**，使用 `/q` 参数跳过回收站，**永久删除所有文件。**
- **Windows路径解析漏洞（2026年1月）**

- AI因未正确处理含空格的路径（如 `obsidian vault`），导致删除指令作用域逃逸，**清空整个系统盘**

人工智能无法担责，最终还是责任在负责操作AI的人，所以在开发中AI提示用户确认授权执行的存在，一般是高危文件和数据库操作，请仔细确认。

Anthropic内部的数据证实了这一点：93%的权限请求被用户直接批准了。

审批疲劳让安全机制形同虚设。Auto模式就是为了解决这个问题。核心思路：用一个AI分类器替你做权限判断。安全操作自动放行，危险操作才拦截。

Plan Mode vs Craft Mode：什么时候用哪个？

Plan 和 Craft 不是非此即彼的选择，而是针对不同场景的两种协作模式。理解它们的差异，能让你的开发效率翻倍。

维度	Plan Mode	Craft Mode
工作方式	先规划后执行，在编码前明确方案	直接执行，快速响应指令
适用场景	复杂功能、架构设计、多文件协同	局部修改、单文件优化、Bug 修复
输出形式	完整方案（需求+技术+设计+任务）	直接代码结果
可控性	高 - 执行前可审阅和调整方案	中 - 边执行边调整
扩展能力	智能编排 MCP/Skill/SubAgent	按需调用扩展

Git操作

这里我们只讨论使用vibe coding过程中的Git操作，大家自行搭建各自项目的代码仓库。

重要提醒：使用Vibe Coding开发一定要使用git工具管理你的代码，并且使用分支管理项目迭代的版本，因为你不知道AI具体改了哪些代码，怎么改的，如果AI的某一次修改或者迭代，不满意想回退之前的代码，这个操作让AI还原他之前的代码，错误很高。

- 1.使用AI Agent帮你进行版本管理，代码提交，commit消息编写。
- 常用的git代码版本库操作直接对AI进行下达即可，比如拉取代码，新建分支，提交代码等。
- 2.AI Agent对Git的理解不只是帮你跑 git 命令。它真的知道你项目当前的版本控制状态，知道你改了哪些文件、在哪个分支上。

一句话commit和PR

最常用的操作：

```
# 让AI自己看看改了什么，写一个commit message
提交当前的改动，写一个有意义的commit message

# 或者更具体一点
commit这些变更，描述清楚我们添加了RSS抓取功能

# 直接创建PR
创建一个PR，标题和描述写清楚这个功能的作用
```

AI Agent会分析你的代码变更，生成一个描述性的commit message，然后执行 git add + git commit。创建PR时它还会自动生成PR描述，包括改了什么、为什么改。

会话管理

别让上下文变成垃圾场, AI开发工具都有上下文限制。对话越长, Agent对当前任务的注意力越分散。用好vibe Coding, 会话管理这件事比你想象的重要得多。

核心会话管理操作

其中会话命名关键字只对应CLI的AI开发工具

清空当前会话: 切换到完全不相关的任务时, 创建一个新的对话, `/clear`;

压缩上下文: 会话太长、AI开始变慢或遗忘, ; `/compact`

停止当前操作: AI Agent在做你不想要的事, 点击停止按钮; `Esc`

回滚: 连按两次 `Esc` 或运行 `/rewind` 打开回退菜单, 恢复到之前的对话和代码状态 (CLI), 或者输入 "撤销刚才的改动"

恢复指定会话: CodeBuddy Code 会把对话保存在本地。当一个任务跨越多次工作时 (比如开始做一个功能, 被打断了, 第二天回来继续), 您不必从头解释上下文:

```
codebuddy --continue    # 继续最近的对话
codebuddy --resume      # 从历史会话中选择
```

用 `/rename` 给会话起个有意义的名字 (比如 "支付重构"、"内存泄漏排查"), 方便以后找。像管理 git 分支一样管理会话——不同的工作线可以有各自独立、持久的上下文。

侧链提问: 想问个不相关的问题, 不污染当前上下文。开启一个ASK的对话, 或者使用 `/btw`

`/clear` 的使用时机

这个命令比你想象的更重要。 `/clear` 会清空当前会话的所有对话历史, 回到一个干净的起点。AI开发平台启动时读取的项目配置.md和项目文件不受影响。

什么时候该用? 当你要开始一个和之前对话完全不同的任务时。

比如你刚才在修一个API的bug, 现在想让AI帮你写一个新的前端组件。如果不clear, AI Agent的上下文里还残留着大量关于那个API bug的信息, 会干扰它对新任务的理解。

推荐 : 修完API bug → `/clear` → 开始前端组件任务。

不推荐 : 修完API bug → 直接说「接下来帮我做个前端组件」→ AI可能把API的上下文混进来

`/compact` 和 `/btw` 的妙用

`/compact` 不是清空对话, 而是让AI Agent把当前对话压缩成一个摘要。适合在一个长会话中途使用: 你和AI已经讨论了很多, 上下文太长影响了性能, 但你不想丢掉讨论的结论。 `/compact` 会保留关键信息, 释放上下文空间。

`/btw` 是一个容易被忽略但非常实用的命令。它开启一个「侧链」对话: 你可以问AI一个和当前任务不相关的问题, 问完之后侧链结束, 主对话的上下文不受影响。

比如你正在让Claude重构一段代码, 突然想问「TypeScript的 Record 类型怎么用来着?」用 `/btw` 问, 不会污染重构任务的上下文。

六个坑, 新人大概率会踩

聊聊使用vibe coding时最常见的错误。这些坑我自己踩过，社区里更是老生常谈问题。

坑1：一个会话什么都塞

修bug、加功能、重构代码、写文档，全在一个会话里做。上下文被塞满，AI对每个任务的理解都很浅。

一个会话聚焦一个任务。做完就 /clear，或者开新对话窗口。

坑2：反复纠正，越改越偏

AI做错了一步，你纠正；改了又错另一个地方，再纠正；第三次还是不对。你花在纠正上的时间比自己动手还多。

纠正两次不行，果断 /clear 重来。重新描述需求，这次说得更具体。在一个已经跑偏的对话上修补，远不如推倒重来。

坑3：看着像对的就接受了AI写了一大堆代码，输出看着挺合理，你就接受了，没实际跑一下。过几天发现边界情况的bug。

每一轮改动都实际运行一次。「代码看起来对」和「代码是对的」差距可能很大。技巧就是：给AI一种验证工作的方式。你自己也一样。

坑4：过度微操

AI每写一个文件你都要看、每改一行代码你都要评论。结果你和AI开发都很慢，而且你其实在用Vibe coding做传统编程。

关注结果。让AI把一个完整任务做完，看最终输出是否符合预期。中间过程除非明显跑偏，不用管。

坑5：需求模糊，然后怪AI不懂你

「帮我优化一下这个代码」「让这个页面好看点」。AI只能猜，而它猜的方向很可能不是你想要的。

给具体的、可验证的需求：「把这个API的响应时间从2秒优化到500ms以内，瓶颈在数据库查询，考虑加缓存或优化SQL」。越具体，输出越接近预期。

坑6：不写项目配置.md

项目根目录没有项目配置.md，或者有但从不更新。每次新会话都要重新解释项目背景、代码规范、技术选型。（每个AI开发工具的配置文件的md，名称都一样，一般是开发工具名称.md的文件）